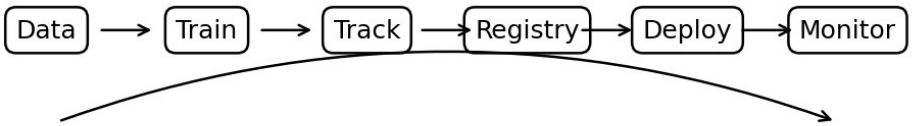
41. MLOPS

Experiment Tracking • MLflow • Registry • CI/CD/CT • Versioning • Monitoring

ML yaşam döngüsü



MLOps = model yaşam döngüsünü tekrar üretilebilir ve gözlemlenebilir hale getirmek

Bu modülün amacı
Bir ML modelini notebook sonucundan yönetilebilir bir production varlığına dönüştürmek: hangi veri ve kodla eğitildiğini bilmek, deneyleri izlemek, modeli registry'de versiyonlamak, otomatik test/deploy/retrain süreçleri kurmak ve production davranışını ölçmek.

Kazanım	Modül sonunda
Experiment tracking	Parametre, metric, artifact ve run metadata'sını sistematik tutabilirsin.
Versioning	Code-data-model-config lineage kurabilirsin.
Registry	Model version/stage/approval yaşam döngüsünü anlayabilirsin.
Automation	CI/CD/CT pipeline tasarlayabilirsin.
Deployment	Batch/online/canary rollout süreçlerini MLOps açısından konumlandırabilirsin.
Monitoring	Service, data, drift, model ve business KPI katmanlarını birlikte izleyebilirsin.

1. MLOps nedir?

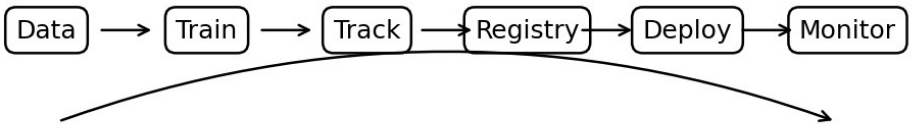
MLOps, machine learning sistemlerinin geliştirme, test, deployment, monitoring ve yeniden eğitim süreçlerini yazılım mühendisliği ve operasyon prensipleriyle yönetme disiplinidir.

ML geliştirme sorunu	MLOps cevabı
Hangi run daha iyiydi?	Experiment tracking
Bu model hangi data ile eğitildi?	Data lineage/versioning
Production'da hangi model var?	Model registry
Deploy manuel	CI/CD pipeline
Model ne zaman bozuldu?	Monitoring/drift
Ne zaman retrain edilmeli?	CT / retraining policy

MLOps = tool listesi değil
MLflow, Docker veya Airflow tek başına MLOps değildir. Asıl konu kontrollü yaşam döngüsü ve tekrar üretilebilirliktir.

2. ML yaşam döngüsü

ML yaşam döngüsü



MLOps = model yaşam döngüsünü tekrar üretilebilir ve gözlemlenebilir hale getirmek

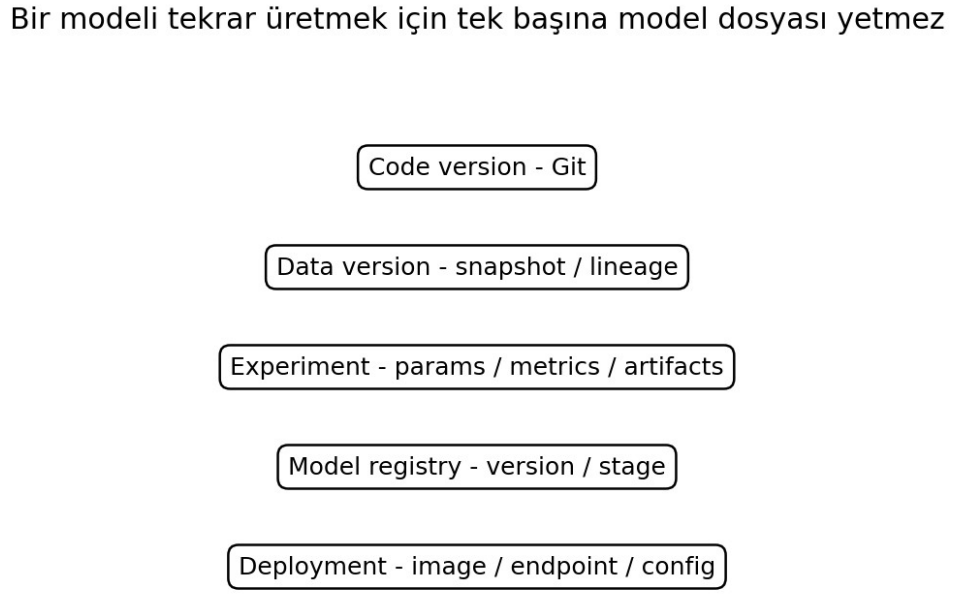
- Data hazırla
- Train
- Evaluate
- Track
- Register
- Deploy
- Monitor
- Retrain

Döngü
Production feedback bir sonraki training iterasyonunun girdisidir.

3. Notebook'tan production'a kopukluk

Notebook	Production
Manuel hücre çalıştırma	Deterministic job
Local path	Versioned storage
Metric ekranda	Tracked metric
model.pkl	Registered artifact
Elle deploy	Pipeline
Manuel kontrol	Monitoring + alert

4. Reproducibility



Bir modeli tekrar üretmek için yalnız model dosyası yetmez.

- Git commit
- Dataset snapshot/version
- Feature definitions
- Hyperparameters
- Library/runtime versions
- Random seed
- Training code

Reproducibility sorusu
6 ay sonra aynı training koşullarını yeniden kurabilir misin?

5. Experiment nedir?

Belirli code/data/config kombinasyonu ile yapılan training/evaluation çalışmasıdır.

Tracked alan	Örnek
Parameters	max_depth=6
Metrics	valid_auc=0.842
Artifacts	model.pkl, plots
Tags	branch=feature/churn
Source	git_sha=abc123
Data	snapshot=2026-10-01

6. Experiment tracking neden?

- Run karşılaştırma
- En iyi modeli bulma
- Audit trail
- Reproduce etme
- HPO sonuçlarını düzenleme

Excel tablosu neden yetmez?
Artifacts, nested runs, source metadata ve otomasyon büyüdükçe manuel tracking kırılır.

7. MLflow mental model

Bileşen	Rol
Tracking	Run params/metrics/artifacts
Models	Model packaging/serving format
Registry	Model version lifecycle
Projects / recipes	Reproducible run patterns

Araç örneği
MLflow çok kullanılan bir açık kaynak örneğidir; aynı kavramlar diğer MLOps platformlarında da vardır.

8. MLflow tracking örneği

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("max_depth", 6)
    mlflow.log_metric("valid_auc", 0.842)

    mlflow.log_artifact(
        "reports/confusion_matrix.png"
    )
```

9. Model loglama

```
import mlflow.sklearn

mlflow.sklearn.log_model(
    sk_model=pipeline,
    name="model"
)
```

Artifact + flavor
Model yalnız binary değil; framework bilgisi, environment ve serving contract metadata'sı da taşınabilir.

10. Autologging

```
mlflow.sklearn.autolog()
model.fit(X_train, y_train)
```

Dikkat
Autolog hız kazandırır ama hangi parametre/metric'in kritik olduğunu bilinçli tasarlamaya yine ihtiyaç vardır.

11. Run naming ve tags

```
mlflow.set_tag(
    "use_case",
    "customer_churn"
)

mlflow.set_tag(
    "git_sha",
    "abc123"
)
```

İyi tag
Run'ı teknik olmayan bağlamla da bulabilmelisin: use case, owner, dataset, branch.

12. Nested runs

Hyperparameter tuning içinde parent run altında çok sayıda child run tutmak düzen sağlar.

Optuna bağlantısı
Her trial ayrı run, study parent run olabilir.

13. Artifact nedir?

- Serialized model

- Feature importance plot
- Confusion matrix
- Training logs
- Evaluation report
- Sample predictions

Artifact store
Local disk yerine object storage kullanmak team/shared environment için daha uygundur.

14. Tracking server mimarisi

Katman	Örnek
Tracking API	MLflow server
Backend store	Postgres
Artifact store	S3/GCS/Blob
Clients	Training jobs/notebooks

15. Data versioning

Training verisinin hangi exact snapshot olduğunu kaydetmek gerekir.

Yöntem	Örnek
Immutable path	data/snapshot=2026-10-01
Table snapshot	Lakehouse version/time travel
Dataset hash	Content fingerprint
Data version tool	DVC/lakeFS tipi yaklaşım

Latest kullanma
training_data/latest.parquet gelecekte aynı dataset olmayabilir.

16. DVC mantığı

Büyük data/model dosyalarını Git dışında storage'da tutup versiyon pointer'larını Git ile ilişkilendirme yaklaşımı.

- Data remote
- Pointer metadata
- Pipeline stages
- Reproduce

Git LFS farkı
İkisi de büyük dosya sorununa yaklaşır; DVC data/pipeline lineage kullanımında daha ML odaklıdır.

17. Data lineage

Bir modelin kullandığı dataset'in hangi upstream kaynak ve transformlardan geldiğini izleyebilmek.

Audit sorusu
Bu feature neden değişti? Hangi upstream tablo değişti?

18. Feature versioning

Risk	Çözüm
Feature logic değişti	Feature definition version
Training/serving farklı	Shared transformation
Historical recompute	Point-in-time snapshot
Owner belirsiz	Feature metadata/registry

19. Model registry nedir?

Model artifact'larını isim/version/stage/metadata ile yöneten merkezi yaşam döngüsü katmanı.

Metadata	Örnek
Name	churn_model
Version	17
Metrics	PR-AUC 0.41
Stage	staging/prod
Data version	snapshot_2026_10_01
Approval	approved_by=ml_lead

20. Registry neden gerekli?

- Production model hangisi?
- Önceki versiyon neydi?
- Rollback nasıl?
- Kim approve etti?
- Hangi metric ile geçti?

Dosya isimleri yetmez
model_final_v7_really_final.pkl kurumsal yaşam döngüsü değildir.

21. Promotion

Model version'ın dev -> staging -> production gibi controlled stage'lerden geçmesi.

Promotion kriteri
Sadece offline metric değil; latency, size, fairness, robustness ve business constraint de gate olabilir.

22. Approval gate

Gate	Örnek
Quality	AUC >= baseline + 0.01
Latency	p95 < 100 ms
Robustness	Edge-case tests pass
Data	No leakage checks pass
Security	Artifact scan pass

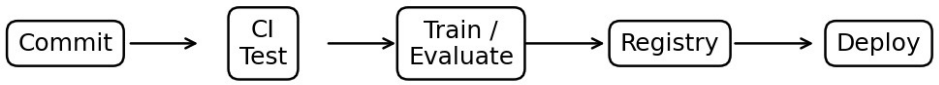
23. Model card

- Intended use
- Training data
- Metrics
- Limitations
- Fairness
- Owner
- Version

Governance bağlantısı
44. Data Ethics & Governance bölümündeki model documentation pratikte registry ile birleşir.

24. CI / CD / CT

ML için CI / CD / CT



CI = code quality | CD = delivery | CT = automated retraining

Kavram	ML anlamı
CI	Code/tests/data-contract checks
CD	Model/service artifact deploy
CT	Data geldiğinde otomatik retrain + evaluate

25. CI içinde neler test edilir?

- Unit tests
- Type/lint
- Schema tests
- Small training smoke test
- Serialization test
- API contract test

Tam training değil
Her PR'da 8 saatlik GPU training gerekmez; small smoke/sanity run yapılabilir.

26. CD pipeline

```
commit
  -> tests
  -> build image
  -> scan
  -> deploy staging
  -> integration tests
  -> approve
  -> deploy production
```

27. Continuous Training (CT)

Yeni data veya trigger geldiğinde training pipeline'ın otomatik çalışması.

Trigger	Örnek
Schedule	Weekly retrain
New data	1M new rows
Drift	PSI threshold
Performance	Recall below target
Manual	Analyst approval

Otomatik retrain = otomatik deploy değil
Yeni model yine evaluation ve promotion gate'lerinden geçmelidir.

28. Training pipeline

```
extract_data
  -> validate_data
  -> build_features
  -> train
  -> evaluate
  -> register
```

DAG
Her task idempotent ve retry-safe tasarlanır.

29. Pipeline orchestration

- Airflow
- Dagster
- Prefect
- Managed cloud pipelines

Araçtan önce kavram
Dependency, retry, schedule, backfill, state, artifacts.

30. Training job isolation

Training job bağımsız environment/container içinde çalışmalı.

- Pinned dependencies
- Pinned image
- Resource request
- Input data version

40. Docker bağlantısı
Container image training runtime'ını stabilize eder.

31. Batch inference pipeline

```
load_features
  -> load_model_version
  -> score
  -> validate_predictions
  -> publish
```

Model version explicit
Production batch job registry'den 'production' alias/version çözmeli; local path'e güvenmemeli.

32. Online serving lifecycle

- Image build
- Model artifact resolve
- Deploy
- Readiness
- Traffic shift
- Monitor
- Rollback

Model + service
Model artifact ve serving code bağımsız versiyonlanabilir; ikisinin uyumluluğu test edilmelidir.

33. Canary deployment

Yeni model/service versiyonuna küçük trafik yüzdesi yönlendirip production sinyallerini karşılaştırma.

İzlenecek	Örnek
Infra	Error rate, latency
Model	Prediction distribution
Business	Conversion/fraud catch
Data	Feature missing rate

34. Blue/Green deployment

Eski ve yeni environment yan yana; traffic bir anda veya kontrollü şekilde yeni tarafa alınır.

Rollback
Traffic'i eski environment'a geri çevirmek hızlıdır.

35. Shadow deployment

Yeni model gerçek request'leri görür ama cevabı kullanıcı kararını etkilemez.

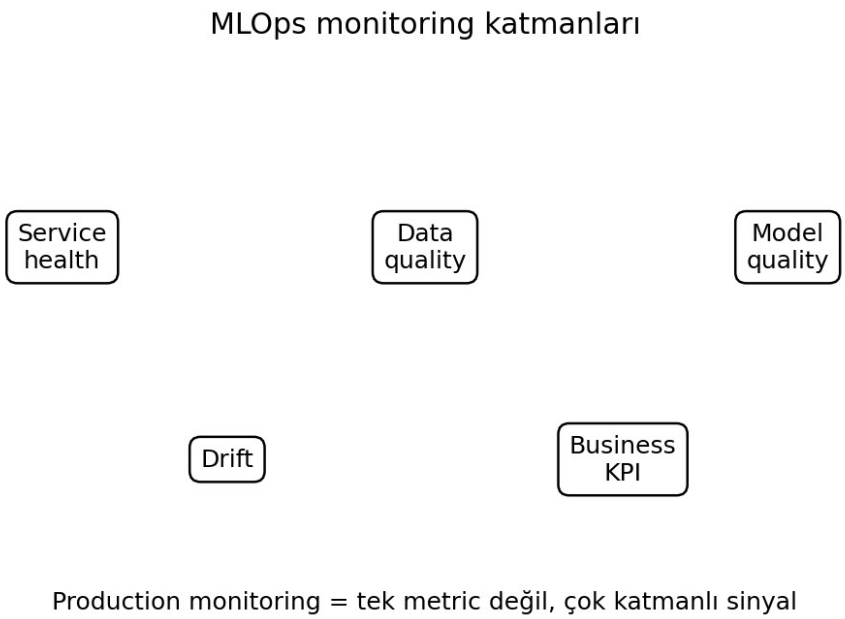
Amaç
Production data üzerinde latency ve prediction behavior karşılaştırmak.

36. A/B testing model releases

Kullanıcılar kontrollü deney gruplarına ayrılarak business impact kıyaslanır.

Offline metric ≠ business metric
AUC artışı conversion veya revenue artışı garanti etmez.

37. Monitoring katmanları



Katman	Örnek sinyal
Service	p95 latency, error rate
Data	null rate, schema, freshness
Drift	PSI, feature distribution
Model	AUC/recall/calibration
Business	Revenue, churn, fraud loss

38. Data quality monitoring

- Schema
- Null rate
- Range
- Category set
- Freshness
- Volume

İlk savunma hattı
Model drift'ten önce input pipeline bozulmuş olabilir.

39. Data drift

Production input feature dağılımının training/reference dağılımından değişmesi.

Ölçü	Kullanım
PSI	Binned distribution shift
KS test	Continuous distribution difference
Jensen-Shannon	Distribution divergence

Category drift	Frequency change

Drift ≠ failure
Dağılım değişimi performans düşüşüyle ilişkili olabilir ama tek başına yeniden eğitim sebebi olmak zorunda değildir.

40. Concept drift

P(y|x) ilişkisinin değişmesi; aynı feature pattern artık farklı target outcome üretebilir.

Label gecikmesi
Concept drift çoğu zaman gerçek label geldikten sonra anlaşılır.

41. Model performance monitoring

- Accuracy/F1/AUC
- Calibration
- Business cost
- Segment metrics

Delayed labels
Fraud/churn gibi use case'lerde label haftalar sonra gelir; leading proxy metric'ler gerekebilir.

42. Prediction drift

Model output dağılımı değişiyor mu? Positive rate veya probability histogram zamanla kayıyor mu?

Alarm
Score distribution değişimi input drift veya model pipeline bug sinyali olabilir.

43. Feature missingness monitoring

Online feature store veya request payload'da missing oranlarını feature bazında izlemek.

Silent bug
Default 0 ile doldurulan missing feature API'yi crash ettirmez ama model performansını bozabilir.

44. Training-serving skew

Sebep	Örnek
Farklı preprocessing	Notebook vs API
Farklı timestamp	Future data leak

Farklı category mapping	Unknown handling
Farklı feature version	Serving eski logic

Çözüm
Shared feature definitions, versioning, point-in-time joins ve online/offline parity tests.

45. Model decay

Model performansının zamanla azalması; drift, user behavior değişimi, product değişimi veya policy etkileri nedeniyle.

Retrain cadence
Takvimle kör retrain yerine evidence-driven cadence daha iyi olabilir.

46. Retraining policy

Policy	Artı / Eksi
Time-based	Basit ama gereksiz retrain olabilir
Data-volume	Yeni bilgi miktarına bağlı
Drift-based	Adaptive ama false alarm olabilir
Performance-based	En anlamlı; label gecikmesi sorun

47. Champion / Challenger

Production champion modele karşı challenger modeli shadow/offline olarak kıyaslamak.

Safe iteration
Challenger açıkça daha iyi olmadan champion değiştirilmez.

48. Rollback

- Previous registry version
- Previous image
- Previous config
- Traffic switch

Rollback test edilmelidir
Sadece teorik olarak mümkün olması yeterli değil.

49. Audit trail

Soru	Cevap nerede?
Kim deploy etti?	CI/CD logs

Hangi model?	Registry
Hangi data?	Data lineage
Hangi code?	Git SHA
Hangi config?	Versioned config
Ne zaman?	Deployment event

50. Metadata

MLOps sisteminin asıl gücü artifact'tan çok metadata graph'ıdır.

- Run -> model
- Model -> data
- Model -> code
- Deployment -> model
- Prediction -> model version

Lineage graph
Incident olduğunda doğru kök nedeni bulmayı hızlandırır.

51. Feature store bağlantısı

Bileşen	MLOps rolü
Offline store	Training features
Online store	Low-latency serving
Registry	Feature definitions
Materialization	Offline -> online sync
Lineage	Feature source tracking

Amaç
Training-serving consistency ve feature reuse.

52. Data contracts

Upstream producer ile ML consumer arasında schema/semantics/freshness sözleşmesi.

Fail early
Feature kolonu type değiştirdiyse production model sessizce yanlış prediction üretmeden pipeline fail etmeli.

53. Model package

- Model binary
- Preprocessing
- Environment
- Signature

- Example input

Model signature
Expected input/output schema registry ve serving validation için değerlidir.

54. Model signature

```
inputs:
  age: float
  income: float
  tenure_months: int

outputs:
  probability: float
```

Contract
Yanlış feature order/type deployment öncesinde yakalanabilir.

55. Model size ve artifact optimization

- Compression
- Quantization
- Pruning
- Format conversion

MLOps trade-off
Daha küçük artifact startup, network ve serving memory'yi iyileştirebilir.

56. Environment versioning

- Python version
- Package lock
- CUDA
- OS/base image
- System libraries

Works on my machine
Container/image digest bu sorunun büyük kısmını çözer.

57. Secrets ve credentials

Tracking server, registry veya cloud storage credential'ları code/experiment metadata içine yazılmamalıdır.

Security
Secret manager/workload identity kullan.

58. Access control

Rol	Yetki
Data scientist	Create run/register candidate
Reviewer	Approve promotion
Deployment service	Read approved model
Admin	Registry policy/manage

Separation of duties
Tek kişinin train + approve + prod deploy yapması regulated ortamda uygun olmayabilir.

59. Cost monitoring

- Training GPU hours
- HPO trial count
- Serving replica cost
- Artifact storage
- Feature compute

Metric
Cost per successful model release veya cost per 1k predictions gibi engineering KPI'lar üretilebilir.

60. Resource reproducibility

Training job'ın CPU/GPU/memory spec'i de experiment metadata'sının parçasıdır.

Benchmark
Aynı model farklı GPU türünde farklı throughput/precision davranışı gösterebilir.

61. HPO MLOps

Her hyperparameter trial'ın run olarak izlenmesi ve best trial'ın registry'ye kontrollü taşınması.

- Search space
- Seed
- Objective
- Trial artifacts

Selection bias
Best validation trial final test set ile yalnız bir kez değerlendirilmelidir.

62. Training validation gates

1. Dataset empty değil.
2. Schema beklenen.
3. Leakage checks geçti.
4. Metric baseline'ı geçti.
5. Prediction finite.
6. Serialization load testi geçti.
7. Inference smoke test geçti.
8. Fairness/segment checks kabul edilebilir.

63. Deployment gates

1. Registry approval var.
2. Image scan temiz.
3. Model signature uyumlu.
4. Latency benchmark geçti.
5. Healthcheck hazır.
6. Rollback version mevcut.
7. Monitoring dashboard hazır.
8. Owner/on-call belli.

64. Incident response

Adım	Soru
Detect	Hangi alarm?
Triage	Data mı service mi model mi?
Mitigate	Rollback/fallback?
Root cause	Lineage ne diyor?
Prevent	Test/gate eklenmeli mi?

Runbook
Production ML sistemi için model/service failure senaryoları yazılı olmalıdır.

65. Fallback strategy

- Previous model
- Rule-based system
- Popularity baseline
- Manual review

Graceful degradation
Model servisi yoksa business tamamen durmak zorunda olmayabilir.

66. Realtime vs batch MLOps farkı

Batch	Online
Job success/freshness	Latency/RPS/availability
Daily output table	Request response
Retry/backfill	Autoscaling/canary
Row-count validation	Tail latency/error rate

67. LLMOps bağlantısı

LLM sistemlerinde klasik MLOps'a prompt, retrieval, vector index ve evaluation set versioning de eklenir.

- Prompt version
- Embedding model version
- Vector index version
- Retriever config
- Golden evaluation set
- LLM/model version

RAG lineage
Cevap kalitesi değiştiyse generator mı retriever mı index mi prompt mu? Versioning bunu ayırmayı sağlar.

68. Prompt registry

Prompt template'larını source code gibi versionlamak ve evaluation sonuçlarıyla ilişkilendirmek.

Production
Prompt değişikliği testsiz hotfix olarak yapılmamalıdır.

69. RAG evaluation pipeline

```
build_index
  -> retrieval_eval
  -> answer_eval
  -> regression_tests
  -> deploy
```

70. MLflow + Docker + Git örnek zinciri

```
git_sha = abc123
docker_image = churn-train:abc123
data_version = snapshot_2026_10_01
mlflow_run = run_789
model_version = churn:17
deployment = prod-2026-10-10
```

Ideal lineage
Bu id'lerin birbirine referans vermesi incident/debugging'i çok kolaylaştırır.

71. Gerçek vaka: churn MLOps

Aşama	Tasarım
Data	Monthly snapshot
Train	Containerized job
Track	MLflow
Gate	PR-AUC + calibration
Register	churn_model:vN
Deploy	Daily batch scoring
Monitor	Score drift + churn capture
Retrain	Monthly + performance trigger

72. Gerçek vaka: fraud MLOps

Aşama	Tasarım
Features	Streaming + online store
Train	Point-in-time historical data
Deploy	Low-latency endpoint
Release	Shadow -> canary
Monitor	Recall, false positives, latency
Fallback	Previous model/rules

73. Gerçek vaka: recommendation MLOps

- Candidate model version
- Ranking model version
- Feature snapshot
- Offline ranking metrics
- A/B experiment
- Business KPI

Multi-model system
Tek model değil, candidate + ranker + reranker zinciri birlikte versiyonlanabilir.

74. Sık yapılan hatalar

- Experiment tracking'i yalnız model metric tablosu sanmak.
- Data version kaydetmemek.
- Production modelini registry dışında dosya path ile seçmek.
- Otomatik retrain sonrası otomatik prod deploy etmek.
- Monitoring'i yalnız CPU/RAM sanmak.
- Drift varsa hemen retrain etmek.

- Rollback planı olmadan deploy etmek.
- Training-serving preprocessing'i ayrı kodlamak.
- Model version ile service image version ilişkisini kaybetmek.
- Approval ve ownership tanımlamamak.

75. Mülakat soruları

Soru	Kısa cevap
MLOps nedir?	ML yaşam döngüsünü reproducible, automated ve observable yönetmek.
Experiment tracking?	Params/metrics/artifacts/source metadata'yı run bazında izlemek.
Model registry?	Versioned model lifecycle ve promotion katmanı.
CI/CD/CT farkı?	Code integration / deployment / automated retraining.
Data versioning neden?	Modelin exact training input'unu tekrar kurabilmek.
Lineage nedir?	Data-code-model-deployment bağımlılık zinciri.
Canary deployment?	Yeni versiyona küçük trafik verip izlemek.
Shadow deployment?	Yeni model gerçek input görür ama kararı etkilemez.
Data drift?	P(x) dağılımının değişmesi.
Concept drift?	P(y|x) ilişkisinin değişmesi.
Training-serving skew?	Offline ve online feature/preprocessing farkı.
Rollback?	Önceki güvenli model/service versiyonuna dönüş.
Champion/challenger?	Prod model ile aday modeli kontrollü kıyaslama.
Model signature?	Input/output schema contract.
Retrain trigger?	Schedule, drift, performance, data volume veya manual.

76. Uygulama alıştırmaları

1. Basit sklearn deneyini MLflow ile logla.
2. Parametre, metric ve artifact kaydet.
3. Git SHA ve data version'ı run tag olarak ekle.
4. İki run'ı karşılaştır ve best modeli seç.
5. Modeli registry benzeri version tablosunda tut.
6. CI için smoke training testi yaz.
7. Batch scoring pipeline DAG tasarla.
8. Canary deployment gate listesi oluştur.
9. Feature drift dashboard metriclerini tanımla.
10. Retraining policy için schedule vs performance trigger karşılaştır.

77. Mini portföy projesi

Proje: End-to-End MLOps Churn Platform

Dosya / katman	İçerik
src/train.py	Deterministic training job
src/evaluate.py	Offline evaluation + gates
mlflow/	Tracking + registry metadata
data/versions/	Dataset version manifest

pipelines/	Train + batch score DAG
Dockerfile	Pinned training/serving runtime
.github/workflows/	CI + image build
monitoring/	Data/model/service metrics
runbooks/	Rollback/retrain incidents
README.md	Lineage + architecture + promotion flow

CV cümlesi
Built an end-to-end MLOps workflow with experiment tracking, data/code/model lineage, containerized training, model registry promotion gates, automated CI pipelines, batch deployment, drift monitoring and rollback-ready versioning.

78. Final cheat sheet

Kavram	Hatırlanacak fikir
Experiment	One reproducible ML run
Tracking	Params + metrics + artifacts
Lineage	Data + code + model relationship
Registry	Model lifecycle/versioning
Promotion	Candidate -> production
CI	Automated code quality
CD	Automated delivery/deploy
CT	Automated retraining
Canary	Small traffic rollout
Shadow	Observe without impact
Drift	Distribution change
Concept drift	Target relationship change
Skew	Training/serving mismatch
Rollback	Return to safe version
Champion/challenger	Controlled model competition
Signature	Input/output contract
Model card	Use, limits, metrics
Monitoring	Service + data + model + business
Runbook	Operational response guide
Reproducibility	Rebuild exact training context

Bu modülden sonra
42. Deployment bölümünde registry'den çıkan modelin batch, API ve cloud serving ortamlarına nasıl güvenli biçimde dağıtıldığını ayrıntılı işleyeceksin.